HomeService OS

Member Task Plan — 4 Members

Member 1 — Backend & Database

1. **Step 1 — Planning (Phase 1)**
   Write requirements for Customer, Home, Appliance, Move Mode AND Services + Technicians (registration, skills, availability, service catalog).
2. **Step 2 — Backend Routes (Phase 3)**
   Build routes/auth.py (customer part), routes/customer.py, AND routes/technician.py, routes/services.py.
3. **Step 3 — Database (Section 14)**
   Create and own: customers, homes, appliances, move_requests, move_appliances AND technicians, services, technician_skills, technician_availability.
4. **Step 4 — Core Product (Phase 4)**
   Implement home/appliance CRUD, multiple-home support AND technician registration/profile, service catalog seeding (Electrical, Plumbing, AC Service, etc. with price/skill/duration).
5. **Step 5 — Move Mode Backend (Phase 6 / Section 9)**
   Build the new-address + appliance-transfer logic; pass appliance list forward for required-service matching (built by Member 3).
6. **Step 6 — Service History (Section 11)**
   Store appliance-specific service history and rule-based maintenance reminders.
7. **Step 7 — Testing (Phase 8)**
   Test customer/home/appliance flows, technician registration, and service catalog data.
8. **Step 8 — Deployment (Phase 9)**
   Deploy customer, home, appliance, technician and service backend routes.

Member 3 — Backend & Database

1. **Step 1 — Planning (Phase 1)**
   Write requirements for Admin, Statistics, Payments, Reviews, Notifications AND Booking + Nearby Matching (status flow, matching rules).
2. **Step 2 — Backend Routes (Phase 3)**
   Build routes/admin.py, services/statistics.py AND routes/booking.py, routes/move.py (matching part), services/technician_matching.py.
3. **Step 3 — Database (Section 14)**
   Own: users, payments, reviews, complaints, notifications, maintenance_reminders AND service_requests, bookings, service_history (technician-facing side).
4. **Step 4 — Core Product (Phase 4)**
   Admin management of customers/technicians/services/bookings/complaints, payment module AND booking status flow (Pending → Assigned → Accepted → Scheduled → In Progress → Completed → Cancelled).
5. **Step 5 — Nearby Matching (Phase 5 / Section 8)**
   Build city/pincode matching, then lat/long distance calculation combined with skill, availability, rating and price.

6 **Step 6 — Move Mode Backend (Phase 6 / Section 9 & 10)**
Identify required removal/installation services per transferred appliance, find technicians near the new home, and build the cost-estimation logic.

7 **Step 7 — Statistics (Phase 7 / Section 13)**
Use Python/Pandas/NumPy/SciPy for customer, technician and admin analytics (mean, median, mode, variance, std. dev., quartiles, correlation, regression, hypothesis tests).

8 **Step 8 — Testing (Phase 8)**
Test admin controls, payments, reviews, booking transitions, matching accuracy, statistics calculations and security.

9 **Step 9 — Deployment (Phase 9)**
Deploy database, admin/analytics, booking and matching backend to production.

Member 2 — Frontend UI/UX

1 **Step 1 — Planning (Phase 1)**
Contribute UI requirements/wireframes for Services, Technicians, Matching, Booking, Payments and Reviews screens.

2 **Step 2 — UI Foundation (Phase 2)**
Follow the shared design system (built with Member 4) when building these screens: colors, buttons, cards, forms, navbar must stay consistent.

3 **Step 3 — Services & Technicians UI (Phase 4 / 5)**
Build the service catalog/browsing pages, service detail cards, and technician profile/listing pages.

4 **Step 4 — Booking UI (Phase 4)**
Build the booking request flow and the status-tracker UI (Pending → Assigned → Accepted → Scheduled → In Progress → Completed).

5 **Step 5 — Nearby Matching UI (Phase 5)**
Build the technician list/map view showing distance, skill match, rating and price clearly.

6 **Step 6 — Move Mode UI — part 1 (Phase 6)**
Build the required-services and nearby-technician-search steps of the Move Mode screens.

7 **Step 7 — Payments & Reviews UI (Phase 4 / Section 12)**
Build the payment summary UI and the post-completion rating/review UI components.

8 **Step 8 — Testing (Phase 8)**
Test these screens for responsiveness and correctness once Member 3 wires up the backend data.

9 **Step 9 — Deployment (Phase 9)**
Optimize and deploy the static assets for these screens; verify rendering in production.

Member 4 — Frontend UI/UX

1 **Step 1 — Planning (Phase 1)**
Contribute UI wireframes for Customer, Home, Appliance, Move Mode, Admin and Statistics screens (Section 1).

2 **Step 2 — UI Foundation (Phase 2)**
Own the shared design system (colors, typography, buttons, cards, forms, modals, alerts, navbar, sidebar, footer) and build the landing page, login/registration and dashboard layouts; hand the system to Member 2 as the shared reference.

3 **Step 3 — Frontend Structure (Section 15)**
Set up templates/, static/css/, static/js/ and base layouts feeding into app.py.

4	**Step 4 — Customer/Home/Appliance UI (Phase 4)** Build the home list/detail UI, appliance cards, and add/edit forms with warranty fields.
5	**Step 5 — Admin & Statistics UI (Phase 4 / 7)** Build the admin dashboard (users, technicians, services, bookings, complaints) and the statistics cards, charts, tables and filter UI for Customer/Technician/Admin analytics.
6	**Step 6 — Move Mode UI — part 2 (Phase 6 / Section 10)** Build the new-address entry, appliance-selection and cost-estimate/booking-confirmation steps of the Move Mode screens.
7	**Step 7 — Consistency Review** Review Member 2's screens against the shared design system; keep visual consistency across the whole app.
8	**Step 8 — Testing (Phase 8)** Test responsiveness, cross-browser rendering and overall UI/UX consistency across all workflows.
9	**Step 9 — Deployment (Phase 9)** Optimize and deploy static assets for these screens; verify the full UI in production; lead final integration checks.

All 4 Together

- **Phase 1 (Planning):** finalize objective, roles, ER diagram, database design and wireframes as one team.
- **Phase 8 (Testing):** full end-to-end workflow testing (login → booking → payment → Move Mode).
- **Phase 9 (Deployment):** final production launch, environment variables, security checks.
- **Later (Section 17):** once the MVP is stable, all 4 contribute to the AI Agent — Member 1 & Member 3 build the controlled backend tools (get_home(), find_nearby_technicians(), create_booking(), etc.), Member 2 & Member 4 design and build the AI chat interface UI.